

Architecture Recap

What the system does

This project learns **station-level correction offsets** for coarse weather inputs.

Instead of predicting temperature directly, it predicts:

- $\Delta T_{\max}$
- $\Delta T_{\min}$

Final corrected values are:

- $\text{corrected_tmax} = \text{ERA5_tmax} + \Delta T_{\max}$
- $\text{corrected_tmin} = \text{ERA5_tmin} + \Delta T_{\min}$

The current focus is daily $T_{\max}$ and $T_{\min}$ correction from ERA5 to station observations.

Data pipeline

Each training sample is **one day over the full station network**.

- Input file: `merged.csv`
- One node = one weather station
- Current network: 23 stations
- Graph type: static directed k -NN graph
- Current setting: $k = 3$

Node features

Each station node uses 6 features:

1. `mx2t` (ERA5 Tmax-like input)
2. `mn2t` (ERA5 Tmin-like input)
3. `UG_era5`
4. `height`
5. `sin_doy`
6. `cos_doy`

Targets

The model is trained on offsets:

- $\text{offset_tmax} = \text{station_TX} - \text{ERA5_mx2t}$
- $\text{offset_tmin} = \text{station_TN} - \text{ERA5_mn2t}$

Edge features

Each directed edge stores 4 spatial features:

1. distance in km
2. Δlat
3. Δlon
4. Δheight

Important detail: missing station targets are **masked out of the loss**, but those nodes stay in the graph so they still provide spatial context during message passing.

Model architecture

There are two main model variants.

1. Baseline model

Pipeline:

node features -> NodeEncoderMLP -> TransformerConv blocks -> OutputHeadMLP -> $[\Delta T_{\text{max}}, \Delta T_{\text{min}}]$

Main structure:

- Node encoder maps 6 input features to a hidden space
- 2 graph Transformer layers perform message passing across stations
- Residual connections and LayerNorm are used after each graph layer
- Output head maps hidden features to 2 values: $\Delta T_{\text{max}}, \Delta T_{\text{min}}$

This is the simpler model, and it is currently the strongest completed baseline in the repo.

2. Multi-channel model

This keeps the same encoder/output-head idea, but replaces the standard graph attention block with **domain-aware channel attention**.

Channels:

- temperature: $[\text{mx2t}, \text{mn2t}]$
- pressure: $[\text{UG_era5}, \text{sin_doy}, \text{cos_doy}]$
- terrain: $[\text{height}]$

How it works:

- each channel has its own attention mechanism
- each channel aggregates neighbor information independently
- channel outputs are concatenated
- a fusion layer projects them back to the shared hidden dimension
- the model then predicts $[\Delta T_{\text{max}}, \Delta T_{\text{min}}]$

Important implementation detail: in the current code, the multi-channel attention uses the **raw node features** for channel slicing and does **not use edge features inside the attention block**.

Training and evaluation

Current training setup:

- Train/validation years: most recent 4 pre-test years, currently 2020-2023
- Held-out test starts at 2024
- Loss: masked MAE on $\Delta T_{\max}$ and $\Delta T_{\min}$
- Optimizer: Adam
- Early stopping patience: 10

Evaluation modes:

- random
- slobo = spatial leave-one-block-out
- st_lobo = spatial + temporal leave-one-block-out

Important evaluation detail: withheld stations are excluded from the **loss**, but remain in the **graph**.

Inference output

At inference time, the model outputs predicted offsets per station per day:

- pred_offset_tmax
- pred_offset_tmin

These are added back to ERA5 to produce corrected station-level temperatures.

Current practical takeaway

- The architecture is designed for **station-network bias correction**, not full weather generation.
- The baseline graph Transformer is currently performing slightly better than the full multi-channel variant in the completed experiments.
- A physics-informed module exists in `src/models/physics.py`, but it is **not part of the active training/inference pipeline right now**.